Reto 2 · La caja negra: doc de diseño

Jaissel Nicol Pérez Marten · Frontier BOG/26 · 10 oct 2026 · Diagrama de arquitectura adjunto

1. Entendimiento del problema

Qué resolvemos: un reemplazo que, tras `POST /__reset`, responda a secuencias de 1 a 12 requests con el **mismo status y el mismo cuerpo** que el original, errores, rutas inexistentes y autenticación incluidos. Además: una especificación en la que cada afirmación cite el log, y un log de experimentos con hipótesis. El éxito se mide con los 100 tests ocultos (solo 3 corridas) evaluados contra la versión vigente a las 20:00.

Lo que ya muestran los 20 ejemplos: un protocolo de texto `CMD;arg…;control` dentro de `{"m": …}`. Parece un mini core bancario (`PING, ALTA, CONSULTA, DEPOSITO, TRANSFER, ANULA`) con familias de error E2xx (integridad), E3xx (sintaxis), E4xx (inexistente) y E5xx (negocio), y con estado por token (contadores `AC-000N` y `OP-000N`).

Supuestos (a validar con el log):

- **S1** El último campo es un dígito de control calculado sobre el resto del mensaje (`ALTA;Lucia;83` falla y `;82` pasa). Es la primera puerta: sin él, nada funciona.
- **S2** Los contadores AC/OP son por token y vuelven a 1 con el reset.
- **S3** `ANULA` revierte saldos: AC-0002 vuelve a 40000 y luego falla una transferencia de 50000.
- **S4** TRANSFER tiene un límite. 120000 falla y 100000 pasa: ¿es por operación o acumulado? Está por probar.
- **S5** El servicio cambia durante el día (hay un incidente anunciado a las 15:30).

Casos difíciles:

- El **orden de evaluación** cuando un mensaje dispara varios errores (E201/E202/E301/E302/E401/E5xx).
- La respuesta sin token, con token ajeno, con cuerpo vacío o no-JSON, y en rutas o métodos inexistentes.
- Montos en 0, negativos, decimales o enormes. Transferencias a la misma cuenta. Anular algo inexistente.
- Rarezas que hay que copiar **literalmente**: `"recieved"`, `cuentaId` vs `cuenta_id`, el espacio final en `"comando desconocido "`, `CONSULTa` aceptado.

Fuera de alcance (y por qué): la persistencia entre reinicios, porque el set hace reset antes de cada test. Los headers, porque no se comparan (pero `X-Req` sí se registra en el log). El escalado horizontal, porque el set le habla a un solo proceso.

2. Arquitectura y trade-offs

Tres componentes. (A) El **explorador** (`enviar.py` + un fuzzer propio): consulta el original con el token personal y escribe cada request en `experimentos.jsonl` con su hipótesis. (B) El **reemplazo**: un servicio FastAPI organizado como pipeline *Auth → Router → Parser → Integridad → Comandos → Estado → Formateador*. (C) El **replayer**: corre las mismas secuencias contra el oráculo grabado y contra el clon, y compara con la misma regla del set. **El clon nunca llama al original.**

Decisión	Elegido	Descartado	Por qué
Lógica del clon	Reglas deterministas en código	LLM dentro del clon	Se exige el texto idéntico carácter por carácter y menos de 5 s. Un LLM no garantiza strings exactos y además cuesta.
Estado	Diccionarios en memoria + reset	Postgres / Redis	Hay un solo proceso y un reset antes de cada test. Una DB suma latencia y otro punto de falla, sin beneficio.
Errores y textos	Una tabla de datos (código, status, mensaje, prioridad)	Strings dispersos en el código	Permite corregir textos y el orden de evaluación en minutos después del cambio de las 15:30.
Versiones	Comportamiento por versión (mañana/tarde) con un flag	Sobrescribir la lógica	Permite documentar el cambio y volver atrás si el diagnóstico estaba mal.
Framework	FastAPI (Python)	Flask, Node/Express	Python ya está en el entorno, el control explícito de status y cuerpo es simple y el catch-all de rutas es fácil.
Despliegue	Railway, 1 réplica, siempre encendido	Túnel local, serverless	Un túnel se cae si se apaga el PC durante el debrief. Serverless reinicia el estado y puede levantar varias instancias.
Exploración	Hipótesis manuales + fuzzer de un carácter	Fuzzing aleatorio masivo	Hay 8.000 req/día y 10 rps. Cada request debe probar algo para que sirva de evidencia en la spec.

3. Plan para medir (antes de construir)

ID	Prueba	Métrica y meta
M1	Replay de los 20 ejemplos contra el clon local	20/20 idénticos antes del primer despliegue
M2	Suite propia de ≥60 secuencias con la forma del set (40% de 1 req, 45% de 2–5, 15% de 6–12), grabadas del original como oráculo	% de pasos idénticos y % de tests completos. Meta ≥90% antes de gastar una corrida del set oculto
M3	Matriz de errores: un mensaje por cada par de errores que dispara ambos a la vez	El orden total de evaluación queda determinado. Cada código E### con ≥2 evidencias pasa a "confirmado"
M4	Regresión: re-ejecutar M2 contra el original cada hora y justo después de las 15:30	0 diffs no explicados. Cada diff queda como fila en "Cambios durante el día"
M5	Uso planificado de las 3 corridas del set oculto	~14:30 línea base · ~18:00 después del incidente · ~19:30 final
M6	Latencia y disponibilidad del entrypoint	p95 < 300 ms (el límite es 5 s). `/health` en 200 durante todo el debrief
M7	Calidad de la spec	100% de las filas citan ≥1 id del log. Cero afirmaciones "confirmado" sin repetición

Riesgos principales: que no se descubra el dígito de control (es la prioridad #1, se ataca con fuzzing variando un carácter a la vez); que el cambio de las 15:30 rompa el clon (se mitiga con la regresión M4 y la tabla de errores como datos); el rate limit (throttle a 8 rps); que la plataforma levante varias instancias (réplicas fijadas en 1).